

■ 日志报错解读手册

排错闭环：先记录时间窗口、服务状态和原始日志；按上下文与退出码定位；修复后重新验证；完成后恢复临时配置并保留处理记录。

打印提示：A4、实际大小（100%）、**关闭浏览器“页眉和页脚”**——本套教材的 PDF 版已自带页码，重复勾选会产生双页码；需要保留彩色底色时，在打印选项里开启“背景图形”。

第 1 页 · 理念篇：日志的构造与读法

1. 日志是什么？

程序运行时会给自己写“工作日记”，这就是日志（log）。**程序死前留下的遗言，就是报错信息**——它其实是程序在告诉你：“我为什么干不下去了，你去看看什么原因。”报错不是敌人，是线索。

2. 一条日志的通用结构

```
Aug 16 08:30:01 ubuntu systemd[1]: Starting Daily apt upgrade and clean activities...
时间戳      主机名    程序名[进程号]    消息内容
```

规律：**时间 + 谁写的 + 说什么**。排错时按时间顺序看，就能还原“事发经过”。

2.1 日志都藏在哪儿？（去哪找线索）

位置	里面是什么
/var/log/syslog	常见系统日志文件，但是否存在取决于发行版与日志服务配置
/var/log/auth.log	常见登录/认证日志文件，路径与名称可能变化
/var/log/nginx/access.log	Nginx 直接安装时的常见访问日志；容器化时可能在容器标准输出
/var/log/nginx/error.log	Nginx 直接安装时的常见错误日志，以实际配置为准
journalctl 命令	查询 journald 日志；systemd 服务优先用 journalctl -u 服务名 确认来源

查找原则：先确认程序、服务和部署方式，再确定日志来源；文件不存在不一定是故障，日志也可能只进入 journald、容器标准输出或自定义路径。

3. 日志级别体系（颜色代表严重程度）

级别	英文	颜色	含义
调试	DEBUG	灰	最啰嗦的细节，排错时看
信息	INFO	白	记录动作或状态；通常不代表错误，也不能仅凭 INFO 判断一切正常，要结合时间线和上下文
警告	WARNING	黄	"有点不对劲，但还能干活"
错误	ERROR	红	"出事了，这件事没干成"
严重	CRITICAL	红加粗	"大事故，我可能要挂了"

注：CRITICAL 是 Linux/通用日志级别；Python logging 标准是**五级**（DEBUG/INFO/WARNING/ERROR/CRITICAL）——《Python 入门》第 10 章为入门只讲四级（到 ERROR），属简化范围，实际项目里 CRITICAL 照样常用。

4. 读日志四步法（核心心法）

- ① 按时间确定故障窗口，先保存原始日志和服务状态
- ② 在窗口内同时查 ERROR、WARNING、INFO、退出码和服务状态，不把某个级别当唯一入口
- ③ 从可疑行向上看前因、向下看后续，并结合配置与上下文验证
- ④ 修复后复现或重跑验证；仍不明白时保留完整原文再查询资料

5. 报错的三种常见句式

✦ 句式①：X + **被动词**（被动式）—— **Permission denied**（权限被拒绝）

✦ 句式②：**cannot / can't** + **动词** + **对象**—— **cannot connect to host**（连不上主机）

✦ 句式③：**No** + **名词**（缺东西）—— **No such file**（没这个文件）

A 组 · Linux 报错 (前 10 条必须熟读)

① Permission denied

逐词: **Permission** (权限) → **denied** (被拒绝, deny 的被动式)

整句: **权限被拒绝 = 你没资格动它**

为什么: 你要读/写/执行一个文件, 但你的身份权限不够

怎么办: **sudo** 借管理员身份; **chmod** 改权限; **chown** 换主人

② No such file or directory

逐词: **No** (没有) → **such** (这样的) → **file** (文件) → **or** (或者) → **directory** (目录)

整句: **没有这样的文件或目录 = 你指的路不存在**

为什么: 路径拼错了、文件名打错了、或你记错了位置

怎么办: **ls** 看看目录里到底有什么; **pwd** 确认自己在哪; Tab 自动补全防拼错

③ command not found

逐词: **command** (命令) → **not** (不) → **found** (被找到)

整句: **命令没找到 = 系统不认识你敲的东西**

为什么: ① 软件没装 ② 命令名拼错了 ③ 程序装了, 但它不在 PATH (系统的"命令搜索路径") 里, 系统不知道去哪找它

怎么办: 先 **which** 命令名 (查命令路径) 看装没装; 没装就 **sudo apt install 软件名**; 装了还找不到 → 重开一个终端 (或 **hash -r** 刷新命令缓存——bash 内置命令, 先了解即可); 用 **echo \$PATH** 看搜索路径; 实在不行就用绝对路径运行, 如 **/usr/local/bin/nginx**

④ No space left on device

逐词: **No** (没有) → **space** (空间) → **left** (剩下) → **on device** (在设备上)

整句: **设备上没有剩余空间 = 磁盘满了**

为什么: 磁盘被写满了, 日志/文件/程序都写不进去了

怎么办: **df -h** 看哪块盘满; **du -sh ~/lab/*** 找谁占的空间 (把 **~/lab** 换成你要查的目标目录, 如 **~/**、**/var/log**; 注意: ***** 不展开隐藏文件, 根目录/家目录建议先 **ls -a** 确认目标, 或用 **du -sh ~/lab/. [^]* ~/lab/*** 补上隐藏文件, 权限不足会报错); 确认文件用途、备份和所属服务后, 再清理明确无用的大文件或旧日志。不要把清空 **/tmp** 当成通用答案, 临时目录中的文件也可能正被程序使用

⑤ Connection refused

逐词: **Connection** (连接) → **refused** (被拒绝)

整句: **连接被拒绝 = 门是关着的 (对方明确拒绝你)**

为什么: 对方的服务根本没启动; 或服务没监听你连的那个端口; 或防火墙直接拒绝 (reject)

怎么办: 确认服务启动没 (**systemctl status 服务名**); 用 **ss -tlnp** 看端口有没有人听; 查防火墙 **ufw status** (**ufw** = 防火墙管理命令, 属于进阶, 先了解即可)

⑥ Connection timed out

逐词：**Connection**（连接）→ **timed out**（超时了：time 时间 + out 用尽）

整句：**连接超时 = 敲了门，等了很久没人应答**

为什么：网络不通；中间有防火墙悄悄丢包；目标机器宕机；路由不通

怎么办：**ping** 目标看通不通；**traceroute** 看断在哪一跳；检查防火墙和安全组

💡 和 refused 的区别：refused = 对方明确说“不”；timed out = 对方连“不”都没说，石沉大海。

⑦ Name or service not known

逐词：**Name**（名字，指域名）→ **service**（服务）→ **not known**（不被知道）

整句：**域名解析不了 = 把网址翻译成 IP 的那一步失败了**

为什么：DNS 服务器连不上；域名拼错；本机 DNS 配置有问题

怎么办：分层排查，不要一步跳到“断网”：① **ip addr** 确认本机地址和网卡状态 → ② **ip route** 确认默认路由 → ③ **cat /etc/resolv.conf** 确认 DNS 配置 → ④ **nslookup** 域名 测试解析。ping 外网 IP（如 8.8.8.8）只是**可选辅助**——它依赖外网、且 ICMP 可能被防火墙禁止，ping 不通 ≠ 断网；能 ping 通 IP 但域名解析失败，才是 DNS 问题的特征。

⑧ Address already in use

逐词：**Address**（地址，指端口）→ **already**（已经）→ **in use**（在使用中）

整句：**端口已被占用 = 这个门牌号已经有人住了**

为什么：你要启动的程序想监听 80 端口，但另一个程序已经占着 80 了

怎么办：**ss -tlnp | grep** 端口号 找到占用的进程；kill 掉它或换一个端口启动

⑨ read-only file system

逐词：**read-only**（只读：read 读 + only 只有）→ **file system**（文件系统）

整句：**文件系统是只读的 = 只能看不能写**

为什么：磁盘硬件出问题系统会自动转只读保护；或人为挂载成只读；U盘写保护

怎么办：先停止可能继续写入的程序，不要直接 remount。先收集 **dmesg**、相关日志、**df -h** 和磁盘状态，判断是否是硬件或文件系统保护。必要时进入维护/恢复环境检查并先保证备份；只有确认原因、目标文件系统和恢复条件后，才考虑高级 remount 分支，不能把它当通用修复。

⑩ Unable to lock the administration directory (/var/lib/dpkg/)

逐词：**Unable to**（无法）→ **lock**（上锁）→ **administration directory**（管理目录）

整句：**无法锁定管理目录 = 有另一个 apt 正在安装软件**

为什么：两个安装命令同时跑，或上次安装中断没清理锁

怎么办（**新手不手动删锁文件**）：锁的作用是防止两个 apt 同时写坏软件数据库。正确顺序：① **ps aux | grep apt** 确认没有 apt 在跑（有就等它结束）；② **sudo dpkg --configure -a** 清理上次中断的残留；③ 重启后仍报锁，查日志找是谁在写。只有在能确认“没有任何 apt 进程、且已用 **dpkg --configure -a** 恢复过数据库”时，才可考虑删除锁文件并立即重建——这是高级操作，新手一律不做。

A 组续 · 其他常见 Linux 报错

⑩ Failed to fetch ... 404 Not Found

逐词: **Failed to fetch** (抓取失败: fetch = 拿取) → **404** (HTTP 状态码) → **Not Found** (没找到)

整句: **下载软件包失败, 404 = 服务器上没这个文件**

为什么: 软件源 (apt 的仓库) 失效、网络中断、或源地址配错了

怎么办: `sudo apt update` 刷新源; 换软件源 (清华/阿里镜像); 检查网络

⑪ E: Sub-process /usr/bin/dpkg returned an error code (1)

逐词: **Sub-process** (子进程) → **returned** (返回了) → **error code** (错误码)

整句: **安装过程中底层工具 dpkg 报错了, 返回错误码 1**

为什么: 软件包冲突、依赖坏了、或上次安装中断留下烂摊子

怎么办: `sudo dpkg --configure -a` 修复未完成的配置; 再 `sudo apt -f install` 修复依赖

⑫ Host key verification failed

逐词: **Host** (主机) → **key** (钥匙/指纹) → **verification** (验证) → **failed** (失败)

整句: **SSH 连接时主机指纹验证失败**

为什么: 第一次连某台机器时记住了它的"指纹", 之后指纹变了 (重装系统/换机器), SSH 怀疑你连的不是同一台, 怕被钓鱼

怎么办: 确认目标是安全的, 删掉旧指纹 `ssh-keygen -R 主机名/IP` 后重连

⑬ Permission denied (publickey)

逐词: **Permission denied** (权限被拒绝) → **publickey** (公钥)

整句: **SSH 免密登录失败: 服务器不认你的钥匙**

为什么: 你的公钥没加到服务器的 `authorized_keys` 里; 或钥匙权限太宽松 (必须是 600)

怎么办 (先保住现有会话, 别把自己锁在门外): ① 保留当前能连的会话不关; ② 把公钥重新拷到服务器 `ssh-copy-id user@host`, 或手动追加到 `~/.ssh/authorized_keys`; ③ 检查权限: `chmod 700 ~/.ssh`、`chmod 600 ~/.ssh/authorized_keys`; ④ 改过 `sshd` 配置先 `sudo sshd -t` 校验语法; ⑤ **新开一个会话验证能登录后, 再关掉旧会话**——顺序反了可能再也进不去。

⑭ Cannot connect to the Docker daemon. Is the docker daemon running?

逐词: **Cannot connect** (连不上) → **daemon** (守护进程: 后台服务) → **running** (运行中)

整句: **连不上 Docker 后台服务: 它没在运行**

为什么: Docker 服务没启动; 或当前用户没权限 (没加入 docker 组)

怎么办: `sudo systemctl start docker` 启动; `sudo usermod -aG docker 用户名` 加组后重新登录 (外部扩展: Docker 的安装、镜像、端口、卷、停止与清理都不在九册前置内, 无 Docker 时本条跳过, 不影响主线)

⑮ port is already allocated

逐词: **port** (端口) → **already** (已经) → **allocated** (被分配)

整句: **Docker 端口已被占用 (和 ⑧ 同一个病)**

怎么办: `docker ps` 看哪个容器占着; 改端口映射 `-p 8081:80`; 或 `ss -tlnp` 找到进程 kill 掉

⑩ Too many open files

逐词：Too many (太多) → open (打开着的) → files (文件)

整句：打开的文件数超过上限

为什么：Linux 每个进程能同时打开的文件数有限（默认 1024），程序忘了关文件或开了太多连接就会触发

怎么办：先看软硬限制：ulimit -Sn (soft 软限制，可临时调高)、ulimit -Hn (hard 硬限制，普通用户只能降不能升)。临时调高只对当前 Shell 及其子进程生效：ulimit -n 65535 (不能超过 hard)；重开终端就失效。长期改 /etc/security/limits.conf 需重新登录生效；程序侧要记得 close。

⑪ Segmentation fault (core dumped)

逐词：Segmentation (内存段) → fault (故障) → core dumped (核心转储：把现场拍下来存了文件)

整句：程序访问了不该访问的内存，当场崩溃

为什么：C/C++ 程序的野指针、越界访问等底层 bug；内存条硬件故障也会

怎么办：升级程序版本；多次出现就查硬件 (memtest 测内存)

⑫ Killed (Out of memory)

逐词：Killed (被杀死) → Out of memory (内存耗尽)

整句：系统内存不足，内核的 OOM Killer 挑了个“最该杀的”进程杀掉

为什么：可能是宿主机内存/交换空间不足，也可能是容器或 cgroup 达到了自己的内存限额；容器被杀时宿主机仍可能有可用内存。

怎么办：宿主机先看 free -h、top 和内核日志；容器再看 docker stats、docker inspect 中的内存限制以及容器状态/事件。宿主机视角的 free/top 不能单独排除容器限额，容器内看到的数值也要结合 cgroup 配置解释；若同时出现 Docker 容器退出，结合本手册 Docker daemon 条目和容器状态一起查。

B 组 · Python 报错 (学 Python 后每天读几条)

💡 Python traceback 的最后一行通常先给出**异常类型和直接原因**，是重要入口，但不是全部证据；具体行号在上方的 File "xxx.py", line 7 中，调用链和前后文也要一起看。常用读法：先看末行知道“错是什么”，再从下往上沿调用链定位“错从哪里传来”。

B 组开篇 · 完整 traceback 逐行拆解 (Python 报错的“全景图”)

Python 出错时打印的那一大段叫 **traceback** (追踪回溯)。看这个两层调用的完整例子：

```
Traceback (most recent call last):
  File "app.py", line 10, in <module>
    main()
  File "app.py", line 7, in main
    helper()
  File "app.py", line 3, in helper
    return 10 / 0
ZeroDivisionError: division by zero
```

逐行拆解 (每行在说什么)：

原文	意思
Traceback (most recent call last):	trace(追踪)+back(回溯)=调用记录。整句：“下面是事故的完整调用链， 最近的调用排在最后 ”——越往下越接近案发现场
File "app.py", line 10, in <module>	File(文件)+line(行)+in(在)+<module>(模块顶层)。意思：案发文件 app.py，第 10 行，位置在顶层代码
main()	第 10 行的那句代码——它调用了函数 main (这是案发的第一层)
File "app.py", line 7, in main	进入 main 函数内部，追踪到第 7 行
helper()	main 里第 7 行调用了 helper (第二层)
File "app.py", line 3, in helper	进入 helper 函数内部，追踪到第 3 行
return 10 / 0	真正的凶手 ——这句代码除以了零
ZeroDivisionError: division by zero	报错类型+信息 = 除以零 (最后一行 = 错误本质)

读法口诀 (从下往上读)：① 最后一行 = 什么错 → ② 往上第一段代码 = 凶手是谁 → ③ 再往上 = 谁叫了它 (调用链，帮你理清来龙去脉)。

① **SyntaxError: invalid syntax**

逐词：**Syntax** (语法) → **Error** (错误) → **invalid** (无效的) → **syntax** (语法)

整句：**语法错误 = 你写的东西 Python 读不懂**

常见原因：冒号忘了、括号没配对、if/for 行尾没加：、用了中文标点

怎么办：看箭头 ^ 指的位置，检查它前面一行；最常漏的是冒号和右括号

② IndentationError: unexpected indent / expected an indented block

逐词: **Indentation** (缩进) → **Error** (错误) → **unexpected** (意外的) → **indent** (缩进); 另一变体 **expected** (期望的) → **block** (代码块)

整句: **缩进错了: Python 靠行首空格区分代码块** (冒号后面的下一行必须缩进, 同一块的缩进必须对齐)

怎么办: 冒号下一行加 4 个空格; 同一层代码缩进对齐; 别把空格和 Tab 混用

③ NameError: name 'x' is not defined

逐词: **Name** (名字) → **Error** (错误) → **is not defined** (没有被定义)

整句: **名字 x 不存在 = 这个变量你还没创建就用它**

常见原因: 变量名拼错了; 变量没赋值就用; 变量定义在函数里却想在函数外用

怎么办: 检查拼写; 确认赋值语句在它之前执行了

④ TypeError: can only concatenate str (not "int") to str

逐词: **Type** (类型) → **Error** (错误) → **concatenate** (拼接) → **str** (字符串) → **int** (整数)

整句: **类型错误: 字符串只能拼字符串, 你拿字符串去拼整数了**

例: "年龄: " + 19 就报这个错; 正确是 "年龄: " + str(19)

怎么办: 用 str()/int()/float() 转换类型再运算; print 里改用逗号或 f-string

⑤ ValueError: invalid literal for int() with base 10: 'abc'

逐词: **Value** (值) → **Error** (错误) → **invalid literal** (无效的字面量) → **base 10** (十进制)

整句: **值错误: 'abc' 不是数字, int() 没法把它转成整数**

怎么办: 转换前先确认内容是数字; 用 try/except 包住转换

⑥ KeyError: 'age'

逐词: **Key** (键) → **Error** (错误)

整句: **字典里没有 'age' 这个键, 你查了个不存在的名字**

怎么办: 用 d.get("age") 代替 d["age"] (get 查不到返回 None 不报错)

⑦ IndexError: list index out of range

逐词: **Index** (索引) → **Error** (错误) → **out of range** (超出范围)

整句: **列表索引越界 = 你要第 10 个, 但列表只有 5 个**

为什么: 列表索引从 0 开始数! 长度 5 的列表, 最后一个的索引是 4, 访问 [5] 就炸

怎么办: 用 len() 先量长度; 循环里用 for x in list 而不是手数索引

⑧ AttributeError: 'list' object has no attribute 'keys'

逐词: **Attribute** (属性) → **object** (对象) → **has no attribute** (没有这个属性)

整句: **属性错误: 列表没有 keys 这个方法 (你把它当字典用了)**

为什么: 对象类型搞错了——keys() 是字典的方法, 列表没有

怎么办: 用 type(变量) 确认类型; 别把 list 和 dict 混着用

⑨ **ModuleNotFoundError: No module named 'requests'**

逐词: **Module** (模块) → **Not Found** (没找到) → **named** (名字叫)

整句: **找不到 requests 这个模块 = 这个库你没安装**

怎么办: `pip install requests`; 装完还是报错就检查是不是装到了别的 Python 版本里

⑩ **ImportError: cannot import name 'xxx' from 'yyy'**

逐词: **Import** (导入) → **cannot import** (无法导入) → **from** (从...)

整句: **导入错误: yyy 模块里没有 xxx 这个东西**

常见原因: 名字拼错; 模块版本太旧没有这个函数; 循环导入 (两个文件互相 import)

怎么办: `dir(yyy)` 看看模块里到底有什么; 升级模块版本

⑪ **FileNotFoundError: [Errno 2] No such file or directory: 'data.json'**

逐词: **File** (文件) → **Not Found** (没找到) → **Errno 2** (系统错误编号 2)

整句: **Python 找不到 data.json 这个文件**

为什么: 文件不在当前目录; 路径写错; 文件名拼错

怎么办: `pwd` 和 `ls` 确认文件在不在脚本所在目录; 用绝对路径最稳妥

⑫ **PermissionError: [Errno 13] Permission denied: 'file.txt'**

逐词: **Permission** (权限) → **Error** (错误) → **denied** (被拒绝)

整句: **Python 打开文件时被系统拒绝 (没权限)** ——和 Linux 的 Permission denied 是同一个病

怎么办: `chmod` 改权限; `sudo` 运行; 别去碰系统目录里的文件 (`/etc`、`/root`)

⑬ **ZeroDivisionError: division by zero**

逐词: **Zero** (零) → **Division** (除法) → **by** (被) → **zero** (零)

整句: **除以零了 = 数学上不允许的操作**

怎么办: 除法前判断分母是不是 0: `if b != 0: a / b`

⑭ **EOFError: EOF when reading a line**

逐词: **EOF** (End Of File: 文件结束) → **Error** (错误) → **when reading** (在读取时)

整句: **还没读完就结束了 = input() 等输入, 但输入流没了**

为什么: `input()` 在没人输入的环境里 (比如在线判题平台) 就会遇到

怎么办: 给 `input` 提供数据; 或不用 `input` 改用参数传入

⑮ **UnboundLocalError: local variable 'x' referenced before assignment**

逐词: **local variable** (局部变量) → **referenced** (被引用) → **before assignment** (在赋值之前)

整句: **函数里用了还没赋值的局部变量 (赋值前就读取了)**

为什么: 函数内给 `x` 赋值 → Python 把 `x` 当局部变量 → 但赋值语句在读取语句后面

怎么办: 把赋值挪到读取前面; 要改全局变量就在函数里加 `global x`

⑮ RecursionError: maximum recursion depth exceeded

逐词: **Recursion** (递归) → **maximum** (最大的) → **depth** (深度) → **exceeded** (被超过)

整句: **递归太深 = 函数自己叫自己叫个没完 (约1000层后 Python 强制叫停)**

为什么: 递归函数忘了写"停下来的条件" (基准条件)

怎么办: 检查递归的出口条件写没写、写得对不对

⑯ JSONDecodeError: Expecting value: line 1 column 1 (char 0)

逐词: **Expecting value** (期待一个值) → **line 1 column 1** (第 1 行第 1 列)

整句: **JSON 解析失败: 想解析的内容根本不是 JSON (第1个字符就不是)**

为什么: API 返回了 HTML 报错页/空内容, 你却当 JSON 解析

怎么办: 先把返回内容 print 出来看看是什么; 确认接口地址和返回格式

⑰ OSError: [Errno 98] Address already in use

逐词: **Address** (地址) → **already** (已经) → **in use** (在使用中)

整句: **端口被占用**——Python 起服务 (如 Flask) 时和 Linux 报错 ⑧ 同一个病

怎么办: 换端口; `ss -tlnp | grep 端口` 找到占用者 kill 掉

⑱ MemoryError

逐词: **Memory** (内存) → **Error** (错误)

整句: **内存不足 = Python 程序吃光内存**

为什么: 一次读入超大文件; 死循环里不停 append 列表; 数据处理量爆炸; 也可能是**系统本身内存不足** (容器/虚拟机限制更常见)

怎么办: 先分清是哪一层——`free -h` 看系统内存是否本来就不够; 再查代码: 大文件改流式 (一行行读, 别 `read()` 一把读); 检查死循环里不停 append; 精简数据结构。若系统内存不足, 是资源问题不是代码问题

⑳ KeyboardInterrupt

逐词: **Keyboard** (键盘) → **Interrupt** (打断)

整句: **你按了 Ctrl+C 把程序打断了 (这不是 bug, 是你干的)**

怎么办: 不用管, 程序是正常的, 你想重启就再运行一次

报错分级速查表（什么颜色代表多严重）

级别	英文关键词	含义	例子	处理
DEBUG	debug	调试细节，最唠叨	进入函数 x()	通常用于定位时查看
INFO	info / notice	过程信息；不等于一定正常	服务启动成功	结合时间线和上下文判断
WARNING	warning / warn	警告，还能用但要注意	磁盘使用率 85%	留个心眼，提前准备
ERROR	error	出错了，这个功能废了	连接数据库失败	要处理
CRITICAL	critical / fatal	严重，系统级故障	磁盘写满了	立刻处理

✦ 处理原则：先按时间线确定故障窗口，再结合 ERROR/WARNING/INFO、上下文、退出码、服务状态和配置定位。ERROR 是重要线索，不是唯一入口；INFO 也可能记录安全、权限或业务异常，不能一律当作“没事”。

实战定位练习（3 个故障现场，先自己想再看答案）

场景 1：SSH 连不上

```
Aug 25 09:30:12 server1 sshd[1024]: Connection closed by 192.168.1.50 port 55900 [preauth]
Aug 25 09:31:05 server1 sshd[1026]: Failed password for invalid user student from
192.168.1.50 port 55920 ssh2
```

问题：① 这两行大概是什么级别的事件？② 发生了什么？③ 第一行的 [preauth] 和 invalid user 说明什么？

场景 2：Python 程序崩溃

```
Traceback (most recent call last):
  File "app.py", line 12, in <module>
    load_config()
  File "app.py", line 8, in load_config
    with open(path, "r") as f:
FileNotFoundError: [Errno 2] No such file or directory: 'config.ini'
```

问题：① 真正的错误是哪一行？② 谁调用了谁（调用链）？③ 怎么修？

场景 3：磁盘满了（以下为模拟日志：主机名、用户名、路径、时间、PID 均为示例，不是真实机器输出）

```
Aug 25 10:00:01 server1 CRON[2001]: (<某用户>) CMD (/home/<某用户>/backup.sh)
Aug 25 10:00:03 server1 backup.sh[2002]: /home/<某用户>/backup.sh: line 3: /backup/data.tar:
No space left on device
```

问题：① 谁是肇事者？② 报错关键词是哪个？③ 处理步骤？

章末作业（3 题，实践题）

✦ 实验约定：① 初始状态：作业 1/3 不产生文件；作业 2 只读不改，无初始文件要求；② 是否依赖前题：三题互相独立；③ 重复运行影响：作业 1 每跑一次产生新 traceback（无残留）；作业 2/3 可无限重复；④ 清理恢复：如为作业 1 创建了临时 .py 文件，完成后删除即可。

1. 自建一个故障现场：跑一段必然会报错的 Python 代码，把完整 traceback 抄下来，用 B 组开篇的拆解方法逐行标注意思。
2. 读自己机器上一个真实存在的日志来源（如 `journalctl -n 20`，或系统中确实存在的 `/var/log/dpkg.log`）最后 20 行，按时间、程序、消息拆解至少 3 条；日志不一定含 INFO/WARNING 字样，不要为了凑级别编造结果。
3. 把 A 组前 10 条的“怎么办”一栏遮住，只看报错原文默写解决办法，然后核对。

尾页 · 排错心法（遇到没见过的报错怎么办）

- ① **别慌，报错是线索不是判决**——程序在告诉你它为什么死，不是骂你
- ② **先读末行，再看调用链**——Python traceback 末行通常给异常类型和直接原因；上方的 File/line 与调用链告诉你它在哪里发生、是谁调用过来的，两部分要一起看
- ③ **先翻本手册 A/B 组找对应条目**；没有就**联网搜索**（外部扩展：把报错原文复制去搜，前面加 `python / ubuntu`；断网时先把原文和上下文完整记录，回网再查——不要离线死猜）
- ④ **一次只改一个变量**——排错时改一处试一次，别一次性改五处（改五处，好了你也不知道是哪处起作用）
- ⑤ **解决后记笔记**——“什么报错 + 什么原因 + 怎么解决”写进学习日志，这以后就是你的《运维踩坑宝典》

日志报错解读手册 · 这不是用来背的，是用来“查”的——遇到就翻，翻多了自然记住

实战练习答案（先做再看）

三个故障现场的答案集中在这里。先自己定位，再对答案；定位错了就把那一段日志重新拆一遍。

场景 1：答案：① 登录失败类事件（比 INFO 严重，算安全告警）。② 有台机器（192.168.1.50）在尝试用 student 这个用户名登录，密码错误。③ [preauth]=还没通过认证就断开了；invalid user=系统里根本没有 student 这个用户——大概率是扫描脚本在批量试弱密码。**边界提醒：第一行 "Connection closed [preauth]" 只能说明"认证前连接被关闭"，单独看它不能推出密码错误；**本例的结论是靠第二行 "Failed password" 推出的。两行结合才构成完整证据。处理：关闭密码登录改用密钥、限制来源 IP。

场景 2：答案：① 最后一行——文件 config.ini 不存在。② 调用链：顶层（第 12 行）→ load_config()（第 8 行）→ open 打开文件时报错。③ 检查路径写没写对，或先创建 config.ini（读法口诀：从下往上读）。

场景 3：答案：① backup.sh（被 cron 定时任务在 10:00 触发）。② **No space left on device**（磁盘满了）。③ 处理步骤：**df -h** 确认哪个分区满 → **du -sh 指定目录/**（先 ls 确认目标目录，**不要在不明确目录直接 du -sh ***：* 不展开隐藏文件，且范围过宽；家目录/根目录要先 ls -a 看隐藏项，或用 du -sh 路径/.[^.]* 路径/* 补齐）→ 确认文件用途、备份和所属服务后，清理明确无用的大文件或旧日志 → 重新跑任务。

章末作业要点提示：作业 1 核对标准=每行都标出“这行是什么”（参考 B 组开篇范例）；作业 2 用理念篇“一条日志的通用结构”来拆（时间+主机+程序+消息），同时记录实际日志来源；作业 3 错一条就回看 A 组对应条目并重新解释原因。